§1.4 Spline Interpolation

Last time we saw that Runge's phenomenon can be fixed by using interpolation points (at the zeroes of Chebyshev polynomials) that aren't equally spaced.

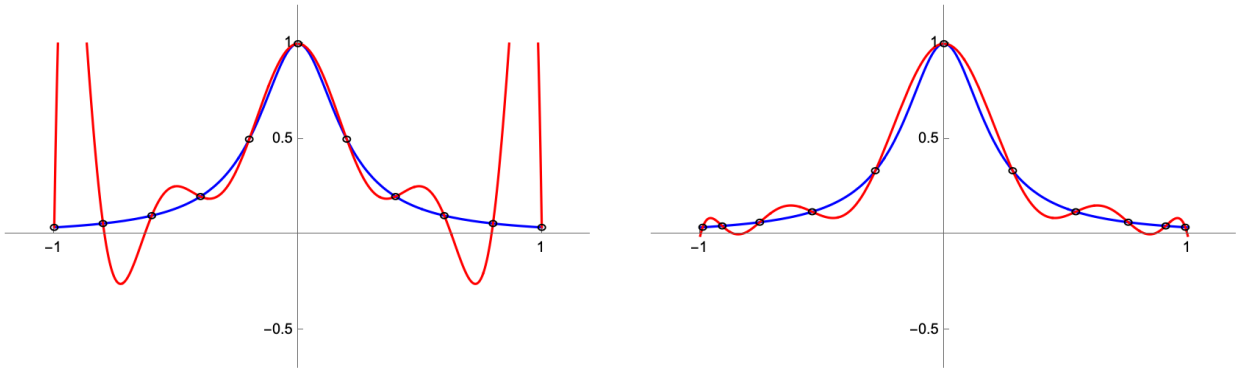

Figure 1.3: Interpolating Polynomial with equally spaced nodes vs. Chebyshev nodes

Another way to improve the fit of the interpolating function is the basis for modern computer aided design (CAD). This approach doesn't require special interpolation points or high-order polynomials, but instead uses pieces of simple polynomials between interpolation points.

Long before computers, ship-builders used thin flexible strips of wood called "splines" to draw curves as they were designing ships. The Romanian mathematician Isaac Jacob Schoenberg is credited with using this same term to describe the piecewise polynomial curves that are now fundamental to computer-aided design. The nice thing about polynomial splines is that they are relatively easy to describe as functions, and yet they can assume an extraordinary range of shapes. Think about trying to come up with a simple function whose graph matches some outline you want. This kind of "inverse problem" is solved very nicely by polynomial splines.

1.4.1 Linear Spline Interpolation

The simplest kind of spline uses linear polynomials to interpolate between each pair of adjacent interpolation points. Figure 1.4 shows images of the linear spline interpolation of Runge's function using 5 and 9 equally-spaced interpolation points. These are clearly better global (i.e. on the entire interval) approximations of Runge's function than the high-order (single) polynomials we saw in Runge's phenomenon.

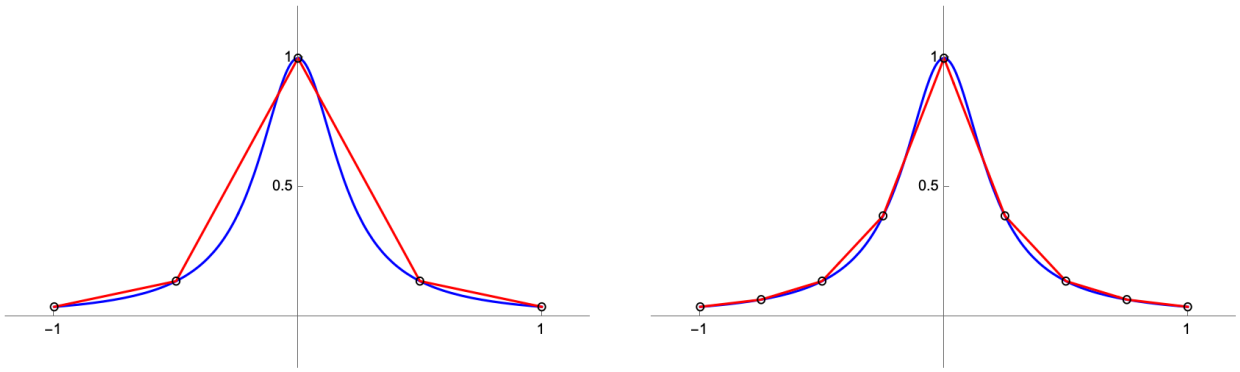

Figure 1.4

Definition 1.4.7: General Formula

Given $n+1$ data points $\{(x_j, f_j)\}_{j=0}^{n}$, we need to construct n linear polynomials $\{s_j\}_{j=1}^{n}$ such that

$$s_j(x_{j-1}) = f_{j-1}, \quad \text{and} \quad s_j(x_j) = f_j$$

for each $j = 1, \ldots, n$. It is simple to write down a formula for these polynomials (compare to question 15)

$$s_j(x) = f_j - \frac{(x_j - x)}{(x_j - x_{j-1})}(f_j - f_{j-1}).$$

Each s_j is valid on $x \in [x_{j-1}, x_j]$, and the interpolant $S(x)$ is defined as $S(x) = s_j(x)$ for $x \in [x_{j-1}, x_j]$.

Accuracy of Linear Splines

Theorem 1.4.8

If $\sigma(x)$ is a function that interpolates $f(x)$ at $n+1$ distinct points $x_0 < x_2 < \ldots < x_n$ and if $\|\sigma'\|_2$ is finite on $[x_0, x_n]$ then,

$$\|S_L'\|_2 \leq \|\sigma'\|_2$$

where S_L is the linear spline interpolating f at the same points. Thus the linear spline is the "flattest" among all splines.

Question 20. Group

Write a careful argument proving the inequality from theorem 8.

One problem with a linear splines can be that they are not necessarily differentiable where two different linear pieces meet. Still, these functions are easy to construct and cheap to evaluate, and can be very useful despite their simplicity. Another problem is that we might need a huge number of linear pieces to reproduce a curved shape. To get around these issues, we can use quadratic or cubic splines instead.

1.4.2 Cubic Splines

The idea behind cubic spline interpolation is to use cubic pieces to interpolate adjacent interpolation points. Cubic functions are defined by 4 coefficients whereas linear functions are defined by only 2.

$$s_i(x) = c_3 x^3 + c_2 x^2 + c_1 x + c_0$$

These extra coefficients allow us to choose cubic pieces that match each other better and/or match the function f better. For instance, we can ensure that the derivatives and the second derivatives of adjacent cubic pieces match at the intersection points, so that the resulting spline will be C^2 everywhere.

The cubic spine requirements can be written as:

$$\begin{aligned} &s_j\left(x_{j-1}\right) = f\left(x_{j-1}\right), && j = 1,\ldots,n; \\ &s_j\left(x_j\right) = f\left(x_j\right), && j = 1,\ldots,n; \\ &s_j'\left(x_j\right) = s_{j+1}'\left(x_j\right), && j = 1,\ldots,n-1; \\ &s_j''\left(x_j\right) = s_{j+1}''\left(x_j\right), && j = 1,\ldots,n-1. \end{aligned} \tag{1.9}$$

■ Question 21. □

Count the total number of requirements (i.e. equations) and the total number of free variables (the coefficients).

What does this tell you about the number of possible solutions to $\mathrm{S}(x)$*?*

So far, we thus have an *underdetermined system*, and there will be infinitely many choices for the function $\mathrm{S}(x)$ that satisfy the constraints.

There are several canonical ways to add two extra constraints that uniquely define S :

- *Natural Boundary Conditions:* requires $\mathrm{S}''(x_0) = \mathrm{S}''(x_n) = 0$;
- *Clamped Boundary Conditions:* specifies exact values for $\mathrm{S}'(x_0)$ and $\mathrm{S}'(x_n)$, or assumes both are zero;
- *Not-a-knot Boundary Conditions:*[‡] requires S''' to be continuous at x_1 and x_{n-1}. This forces $s_1 = s_2$ and $s_{n-1} = s_n$.

Natural cubic splines are a popular choice for they can be shown, in a precise sense, to minimize curvature over all the other possible splines. They also model the physical origin of splines, where beams of wood extend straight (i.e., zero second derivative) beyond the first and final 'ducks.'

■ Question 22. □

Suppose S_3 *is the natural cubic spline interpolant to* $\left\{(x_j, f_j)\right\}_{j=0}^{n}$*, and* σ *is any* $\mathrm{C}^2[x_0, x_n]$ *function that also interpolates the same data. Then show that*

$$\|\mathrm{S}_3''\|_2 \leq \|\sigma''\|_2.$$

Hint: Similar to the proof of theorem 8, first show that

$$\|\sigma''\|_2^2 = \|\mathrm{S}_3''\|_2^2 + \|(\sigma'' - \mathrm{S}_3'')\|_2^2 + 2\int_{x_0}^{x_n} (\sigma'' - \mathrm{S}_3'')\mathrm{S}_3'' \,\mathrm{d}x.$$

Then use integration by parts.

■ Question 23. Lab

In this problem, we will compare the Not-a-knot and clamped boundary conditions.

(a) *Define the function* $f(x) = \sin(20x) + e^{5x/2}$.

[‡]In spline parlance, the interpolation nodes $\left\{x_j\right\}_{j=0}^{n}$ are called *knots*.

(b) *Define a* `list` *of data points* $\{x_i, f(x_i)\}$ *using the* `Table` *command as* x_i *ranges from* 0 *to* 1 *with* 0.1 *increment intervals.*

(c) *Interpolate the data via piecewise-cubic splines, using the Not-a-knot boundary conditions (this is the default).*

In `Mathematica`*, you can use* `ResourceFunction["CubicSplineInterpolation"]`*. See this page for usage examples. Alternately, see here for* `Python`*.*

(d) *Next consider clamped boundary conditions. First, analytically compute the derivative at the two endpoints. Then interpolate the data with these values as the boundary conditions.*

In Mathematica, use `{{"Clamped",f'[0]},{"Clamped",f'[1]}}` *as an optional argument. In Python, we pass a third argument into cubic spline interpolation package, see* `bc_type`*.*

(e) *The absolute error for an interpolant* $p(x)$ *is defined as* $|f(x) - p(x)|$*. Plot the absolute errors for the two boundary condition methods on the same set of axes over the interval* $[0, 1]$*. Which one did a better job of estimating the function near the endpoints? Does this match your expectations? Why or why not?*